WEEK 05

# HTML5/CSS: Advanced Concepts

Front-End Development — Web Applications Development

Undergraduate CS Students • Beginners Level

Cyber.SoHo | March 2026

# Today's Agenda

Seven topics to transform your web pages

| | | |
|---|---|---|
| 01 | **HTML5 Semantic Elements** | Give meaning to your markup |
| 02 | **CSS Flexbox** | One-dimensional layouts |
| 03 | **CSS Grid Layout** | Two-dimensional page structure |
| 04 | **Transitions & Animations** | Motion and interactivity |
| 05 | **Responsive Design** | One site, every screen |
| 06 | **HTML5 Forms** | Built-in validation, no JS needed |
| 07 | **CSS Variables & Theming** | Maintainable, dynamic styles |

01

# HTML5 Semantic Elements

Giving meaning to your markup

# The Problem: 'Div Soup'

Before HTML5, every section was a generic  with an arbitrary class name

Screen readers for visually impaired users saw only anonymous boxes

Search engines had zero structural clues for indexing content

Developers returning to code months later couldn't understand the layout

Analogy: A book where every chapter is titled 'Section' — meaningless!

THE OLD WAY

```
<div id="header">
<div class="nav">
<div id="content">
  <div class="post">
  <div class="post">
<div class="sidebar">
<div id="footer">
```

# The Solution: Semantic Tags

| Semantic Tag | Replaces | Purpose |
|---|---|---|
| <header> |  | Page/section banner |
| <nav> |  | Navigation links |
| <main> |  | Primary content (1 per page) |
| <article> |  | Self-contained content |
| <section> |  | Thematic grouping |
| <aside> |  | Related secondary content |
| <footer> |  | Closing info / copyright |

# Why Semantic Elements Matter

## Accessibility (a11y)

Screen readers (NVDA, JAWS) use semantic tags as landmarks — users can jump directly to <main>, skip repetitive headers

## SEO (Search Engine Optimization)

Search engines reward structured HTML with better rankings — semantic tags act as content signals

## Developer Experience

<nav> is self-documenting.  is not. Semantic code is maintainable code.

KEY TAKEAWAY

# Semantic Elements

Replace structural  tags with HTML5 semantic elements (<header>, <nav>, <main>, <article>, <section>, <aside>, <footer>).

Your pages become accessible, search-engine-friendly, and self-documenting — with zero performance cost.

02

# CSS Flexbox

One-dimensional layouts made easy

# What Is Flexbox?

Flexible Box Layout — a one-dimensional CSS layout model

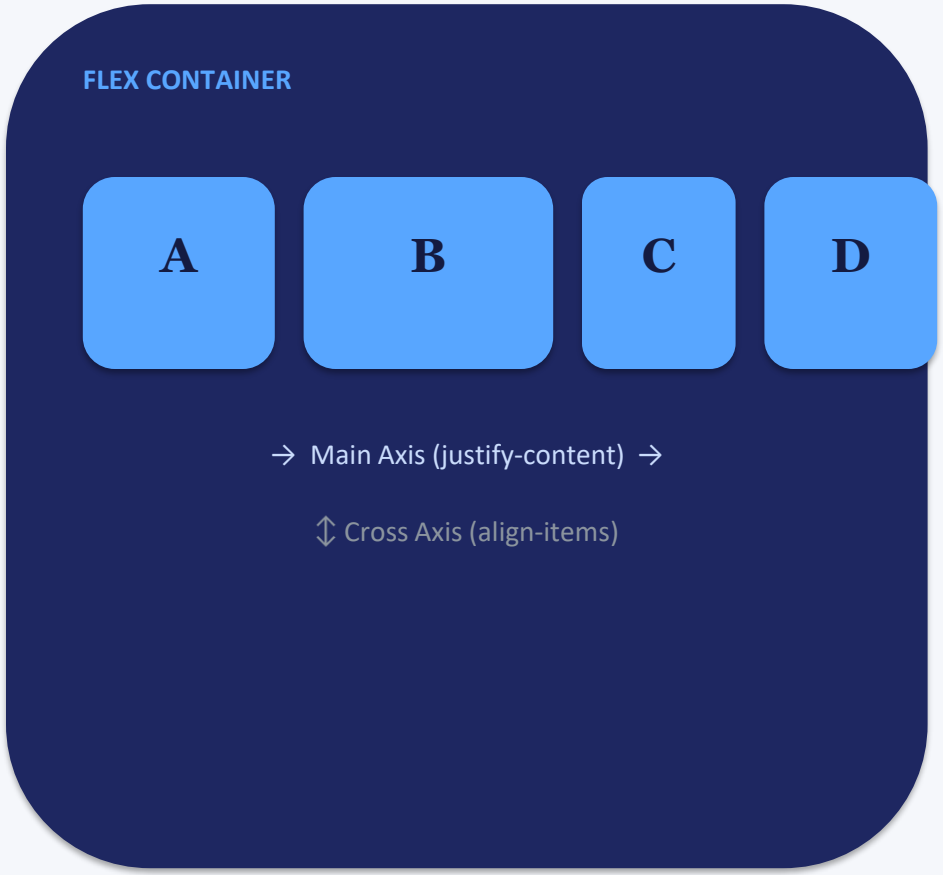
Distributes items along a single axis (row or column)

Replaced old hacks: floats, table layouts, negative margins

W3C Recommendation since 2018 — universal browser support

Two roles: Flex Container (parent) + Flex Items (children)

Activate with one line: display: flex;



# Flexbox Container Properties

| Property | Controls | Common Values |
|---|---|---|
| flex-direction | Main axis direction | row, column, row-reverse, column-reverse |
| justify-content | Main-axis spacing | flex-start, center, space-between, space-around |
| align-items | Cross-axis alignment | flex-start, center, stretch, flex-end |
| flex-wrap | Multi-line wrapping | nowrap (default), wrap |
| gap | Spacing between items | Any length: 1rem, 20px |

# Flexbox Item Properties

| Property | What It Does | Example |
|---|---|---|
| flex-grow | How much extra space item absorbs | 0 = none, 1 = proportional |
| flex-shrink | How much item shrinks when tight | 0 = never, 1 = proportional |
| flex-basis | Ideal starting width | 260px, 30%, auto |
| order | Visual reordering | Lower numbers appear first |

**The Shorthand:**

```
flex: <grow> <shrink> <basis>    →    flex: 1 1 260px  means grow:1, shrink:1, start at
260px
```

KEY TAKEAWAY

# CSS Flexbox

Flexbox handles one-dimensional layouts — rows OR columns.

Activate with display: flex on the parent. Use justify-content (main axis) and align-items (cross axis) for distribution. Use flex-wrap: wrap for responsive card grids. Nest Flexbox containers for complex inner layouts.

03

# CSS Grid Layout

Two-dimensional page structure

# What Is CSS Grid?

A two-dimensional layout system — rows AND columns simultaneously

Purpose-built for full page layouts, dashboards, complex arrangements

W3C Recommendation since 2017 — universal browser support

Not a replacement for Flexbox — they complement each other

### Use Flexbox When...

Content determines size

One-axis flow (nav bar, card row)

Items should grow/shrink freely

### Use Grid When...

Layout determines placement

Full page structure needed

Precise row + column control

# Grid Terminology

| Term | Meaning |
|---|---|
| Grid Container | Parent with display: grid |
| Grid Item | Each direct child of the container |
| Track | A single row or column |
| Cell | Intersection of one row + one column |
| Area | Named rectangular group of cells |
| fr unit | Fractional unit — 'one share of remaining space' |
| Gap (Gutter) | Space between tracks |

# The Grid Workflow — 3 Steps

1 **Define Tracks**

```
grid-template-columns: 260px 1fr 1fr;
grid-template-rows: auto 1fr auto;
```

2 **Name Areas**

```
grid-template-areas:
  "sidebar header  header"
  "sidebar main    stats"
  "sidebar footer  footer";
```

3 **Place Items**

```
.sidebar { grid-area: sidebar; }
.header  { grid-area: header; }
```

KEY TAKEAWAY

# CSS Grid Layout

CSS Grid controls both rows and columns simultaneously.

Define structure with grid-template-columns/rows/areas. Place children with grid-area. Use fr units for flexible sizing. Collapse to single column on mobile with @media queries.

# 04 Transitions & Animations

Adding motion and life to your pages

# CSS Transitions — Smooth State Changes

Smoothly animate a property change when triggered

Requires a trigger: :hover, :focus, JS class toggle

Moves between exactly two states (A → B)

```
transition: <property> <duration> <timing-function> <delay>;
transition: width 0.5s ease-out 0.1s;
```

## Timing Functions

| Function | Behaviour |
|---|---|
| ease | Slow → fast → slow (default) |
| linear | Constant speed |
| ease-in | Slow start |
| ease-out | Slow end |
| ease-in-out | Slow start and end |

# @keyframes — Multi-Step Animations

Define multi-step animations with any number of stops

Run automatically — no trigger needed

Can loop infinitely (infinite) or play once

```
@keyframes pulse {
  0%   { transform: scale(1); }
  50%  { transform: scale(1.3); }
  100% { transform: scale(1); }
}
.el { animation: pulse 2s infinite; }
```

**Animation Shorthand:**

```
animation:
  name
  duration
  timing-function
  delay
  iteration-count
  direction
  fill-mode
  play-state
```

# Transitions vs. @keyframes — Comparison

| Feature | Transition | @keyframes Animation |
|---|---|---|
| Trigger needed? | Yes (hover, focus, JS) | No (auto-plays) |
| Number of states | 2 (start → end) | Unlimited (0%→25%→…→100%) |
| Looping | No | Yes (infinite) |
| Direction control | No | Yes (alternate, reverse) |
| Best for | Hover effects, UI polish | Spinners, attention-getters |

KEY TAKEAWAY

# Transitions & Animations

Use transitions for smooth, triggered state changes (hover effects, reveal animations).

Use @keyframes for multi-step, autonomous animations (spinners, pulsing elements). Both use the same timing functions and transform property.

05

# Responsive Web Design

One site, every screen size

# The Problem: One Website, Many Screens

Users view your site on: 6" phone → 10" tablet → 15" laptop → 32" desktop

A fixed-width page designed for 1920px is unusable on a 375px phone

Same user may switch between devices in a single day

RWD (Responsive Web Design) was coined by Ethan Marcotte in 2010

## The Three Pillars of RWD

### Fluid Grids

% or fr-based sizing

### Flexible Media

Images that scale

### Media Queries

Conditional CSS rules

# Media Queries & Breakpoints

```
@media (min-width: 768px) {
    .container { grid-template-columns: 1fr 1fr; }
}
```

| Breakpoint | Target | Typical Layout |
|---|---|---|
| < 600px | Phone (portrait) | 1 column, stacked |
| 600px – 1023px | Tablet | 2 columns |
| ≥ 1024px | Desktop | 3 columns |
| ≥ 1440px | Large desktop | 3-4 columns, wider content |

# The Mobile-First Approach

Write base CSS for the smallest screen (single column, compact)

Add min-width queries to enhance for larger screens

Extra complexity is layered on — never stripped away

Smallest, slowest devices get the lightest CSS

**THE CRITICAL TAG (without it, mobile responsive CSS is ignored):**

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

**Mobile-First Progression:**

**Base CSS (1 column)** → **@media ≥600px (2 columns)** → **@media ≥1024px (3 columns)**

KEY TAKEAWAY

# Responsive Web Design

Use <meta viewport> + mobile-first CSS + @media (min-width: ...) queries to create layouts that adapt to any screen.

Write for phones first, then add columns and complexity for tablets and desktops. Test with browser DevTools device toolbar (F12).

06

# Advanced HTML5 Forms

Built-in validation — no JavaScript needed

# New HTML5 Input Types

Before HTML5: dozens of lines of JS. Now: the browser validates for you.

| Type | Validates / Renders | Example |
|---|---|---|
| email | Checks for @ and domain | <input type="email"> |
| url | Checks for protocol | <input type="url"> |
| date | Native calendar picker | <input type="date"> |
| number | Spinner + min/max/step | <input type="number" min="1"> |
| range | Draggable slider | <input type="range"> |
| tel | Phone keyboard on mobile | <input type="tel"> |
| color | OS colour picker | <input type="color"> |

# Validation Attributes & Pseudo-Classes

| Attribute | What It Does |
|---|---|
| required | Field cannot be empty |
| pattern="regex" | Must match a Regular Expression |
| min / max | Numeric or date boundaries |
| minlength / maxlength | Character count limits |
| step | Numeric increment (e.g., 0.01) |

## CSS Pseudo-Classes:

| | |
|---|---|
| `:valid` | Value passes all rules |
| `:invalid` | Value fails a rule |
| `:required` | Has required attr |
| `:optional` | No required attr |

**<datalist> — Autocomplete Suggestions**

Not a dropdown (<select>) — users can still type freely. Connected via the list attribute.

KEY TAKEAWAY

# HTML5 Forms

HTML5 input types (email, date, number, url, range, color) and attributes (required, pattern, min/max) let the browser validate forms without JavaScript.

Use :valid and :invalid pseudo-classes for automatic green/red visual feedback. The <datalist> element adds autocomplete for free.

07

# CSS Variables & Theming

Maintainable, dynamic style systems

# CSS Custom Properties (Variables)

Store a value once, reuse it everywhere in your stylesheet

Follow the DRY (Don't Repeat Yourself) principle

Declared with -- prefix, used with var()

Most commonly declared in :root (global scope)

```
:root {
  --brand-color: #6c5ce7;
  --spacing-md:  1.2rem;
}
.button {
  background: var(--brand-color);
  padding: var(--spacing-md);
}
```

**The Power:**

Change --brand-color in ONE place → every element using var(--brand-color) updates automatically.

Used by Bootstrap 5, Tailwind CSS, Material UI — the entire modern CSS ecosystem.

# Dynamic Theming with Variables

```
/* Default (light) */
:root      { --bg: #ffffff; --text: #333; --accent: #6c5ce7; }

/* Dark override — just redefine the same variables */
html.dark   { --bg: #1a1a2e; --text: #ddd; --accent: #e17055; }

/* JavaScript: one line toggles the entire theme */
document.documentElement.classList.toggle("dark");
```

## How It Works:

1. Define default variables in :root
2. Override them in a theme class (html.dark)
3. Toggle the class with one line of JS
4. Every var() reference updates instantly — zero manual repainting

KEY TAKEAWAY

# CSS Variables & Theming

CSS Custom Properties (--name: value / var(--name)) eliminate repetition and enable dynamic theming.

Declare in :root for global scope, override in theme classes, and swap themes with a single class toggle on <html>. Every major CSS framework relies on this pattern.

# Quick Reference — All 7 Topics

| # | Topic | Core Concept |
|---|---|---|
| 1 | Semantic Elements | Tags that describe purpose (<header>, <nav>, <main>) |
| 2 | CSS Flexbox | One-dimensional layout (display: flex) |
| 3 | CSS Grid | Two-dimensional layout (display: grid) |
| 4 | Transitions | Smooth A→B changes on trigger |
| 5 | @keyframes | Multi-step autonomous animations |
| 6 | Media Queries | Conditional rules for screen sizes |
| 7 | CSS Variables | Reusable values and dynamic theming |

# Essential Resources

| Resource | URL | What It Is |
|---|---|---|
| MDN Web Docs | developer.mozilla.org | Comprehensive HTML/CSS/JS reference |
| W3C | w3.org | Web standards body |
| CSS-Tricks Flexbox | css-tricks.com/.../flexbox | Visual Flexbox guide |
| CSS-Tricks Grid | css-tricks.com/.../grid | Visual Grid guide |
| Flexbox Froggy | flexboxfroggy.com | Game to learn Flexbox |
| Grid Garden | cssgridgarden.com | Game to learn Grid |
| Can I Use | caniuse.com | Browser compatibility tables |
| freeCodeCamp | freecodecamp.org | Free interactive curriculum |

THE BIG PICTURE

# Building Modern Web Pages

| | |
|---|---|
| **HTML5 Semantics** | give your content meaning |
| **Flexbox & Grid** | give it structure |
| **Transitions & Animations** | give it life |
| **Media Queries** | make it universal |
| **CSS Variables** | make it maintainable |

**Together, they are the foundation of every modern website.**

DISCLAIMER

# Legal & Attribution

All material is intellectual property of Cyber.SoHo — unauthorised reproduction prohibited

Not affiliated with, endorsed by, or sponsored by any organisations mentioned herein

Verify external links at virustotal.com/gui/home/url before visiting

Provided "as is" for educational purposes — no warranties express or implied

Created with commitment to academic integrity and ethical knowledge sharing

Cyber.SoHo © 2026 • Week 05: HTML5/CSS Advanced Concepts